Büyük Dil Modellerine Giriş ve Temel Kavramlar

Dersin yürütücüsü: Prof. Dr. Banu Diri

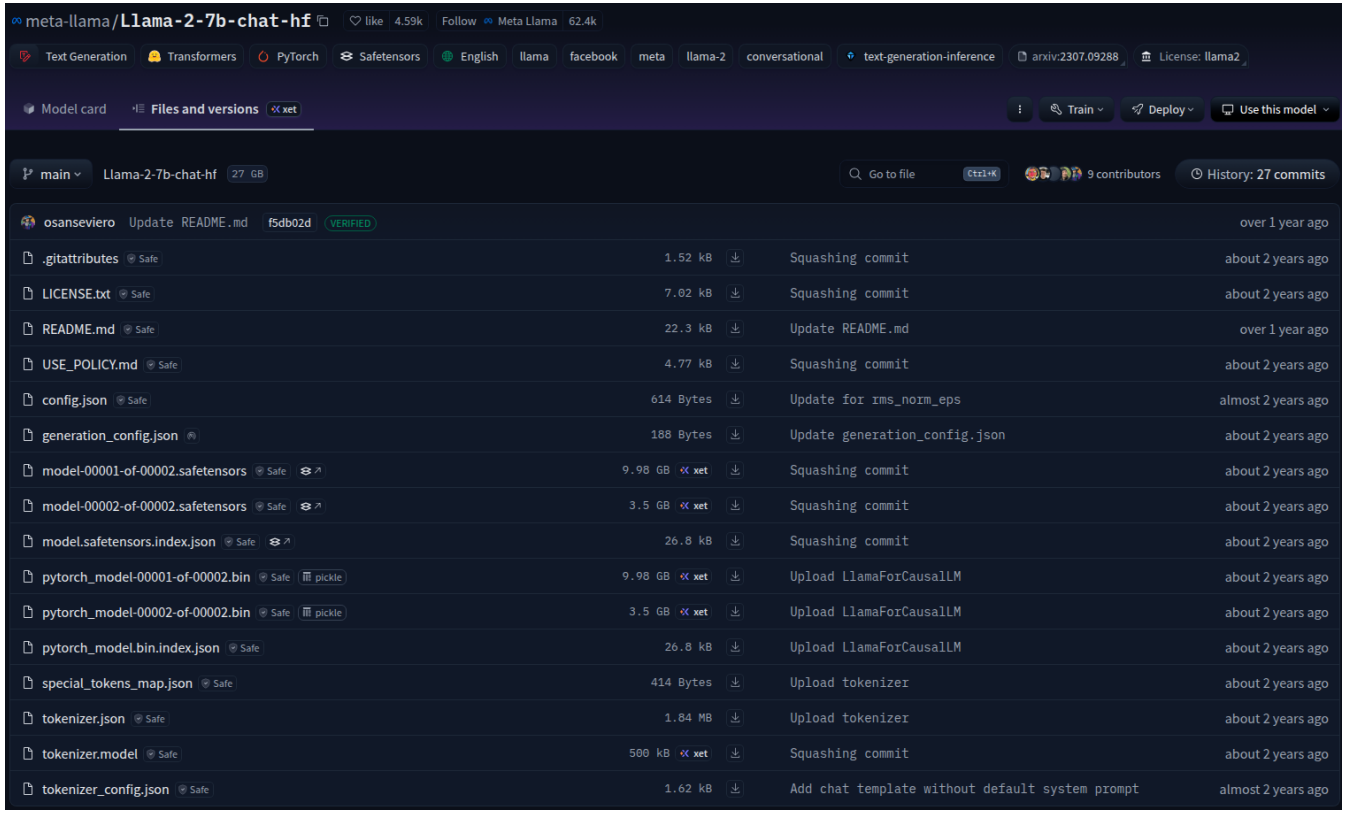
Büyük Dil Modelleri (BDM - LLM) Nedir ve Nasıl Çalışır?

BDM'lere Sezgisel Bir Giriş: Parametreler ve Çalıştırma Kodu

Bir büyük dil modeli uygulamada birkaç farklı dosya türünden oluşur bu dosyalar temelde:

- Model ağırlıkları
- Tokenizer
- Konfigürasyon

dosyalarından oluşur. LLama2 için örneklere bakalım.



Model ağırlık dosyaları

- Model ağırlık dosya(lar)ı (safetensors ya da bin uzantılı dosyalar)
  - Modelin beynini oluşturan verileri içerir. Çeşitli ağırlıklar, parametreler vs.
- model.safetensors.index.json
  - .safetensors uzantılı dosyalar için ağırlık haritalaması (weight mapping) bilgilerini içerir. Hangi parametrenin hangi ağırlık dosyasında olduğu bilgisini tutar.
- pytorch_model.bin.index.json
  - .bin uzantılı dosyalar için ağırlık haritalaması (weight mapping) bilgilerini içerir. Hangi parametrenin hangi ağırlık dosyasında olduğu bilgisini tutar.

Tokenizer dosyaları

- tokenizer.json - ana dosya
  - Bu dosya tokenizer'ın tüm çalışma mantığını içerir.
  - Tüm token'ların listesi ve ID'leri
  - Tokenizer için birleştirme kuralları
  - Metni nasıl böleceği
  - Token'ları nasıl düzenleyeceği
  - Metin normalizasyon kuralları
- tokenizer_config.json
  - Azami token uzunluğu
  - Nasıl dolgu (padding) yapılacağı
  - Uzun metinleri nasıl keseceği
  - Özel token'ların ne olduğu
  - Boşluk temizleme ayarları
- Tokenizer.model
  - Binary biçimde **SentencePiece** tokenizer modeli
  - Vocabulary ve merge kurallarını içerir
- special_tokens_map.json
  - Bu dosya modelin kullandığı **özel token**'ları tanımlar
  - <|begin_of_text|>, <|end_of_text|> vb.

Konfigürasyon dosyaları

- Config.json
  - Modelin temel mimari özelliklerini ve yapısal parametrelerini içerir
  - Model türü (llama, gpt2, bert vb.)
  - Gizli katman boyutu (örn: 4096)
  - Vocabulary boyutu (örn: 32000)
  - …..
- generation_config.json
  - Modelin metin üretirken kullanacağı parametreleri belirler
  - temperature, top_p, top_k, repetition_penalty vb.

Kapalı Modeller

Şirketler tarafından geliştirilen ve ticari sır olarak saklanan modellerdir.

Model ağırlıkları gizlidir.

Sadece şirketin sunduğu API arayüzü ile kullanılabilir.

Modelin nasıl ve hangi verilerle eğitildiği açıklanmaz/bilinmez.

Kullanıcılar modeli özelleştiremez veya ince ayar (fine-tune) yapamaz.

GPT-4 (OpenAI), Claude (Anthropic), Gemini (Google) gibi örnekler verilebilir.

Açık Kaynak Modeller

Herkesin ücretsiz erişebildiği ve kullanabildiği modellerdir.

Model ağırlıklarını herkes indirebilir ve kullanabilir

Kendi yerel sunucunuzda çalıştırma mümkün.

İnce ayar (fine-tuning) yapabilir, modeli değiştirebilirsiniz

Eğitim süreci ve veri kümeleri genellikle belirtildiği için daha şeffaflardır.

Llama serisi (Meta), Mistral, Mixtral, BERT (Google), Falcon, Qwen gibi örnekler verilebilir.

Kapalı - Açık Kaynak Model Kıyaslaması

Özellik	Kapalı Modeller	Açık Kaynak Modeller
Maliyet	API kullanım ücreti	Sadece donanım maliyeti
Gizlilik	Veriler şirkete gider	Tam veri gizliliği
Özelleştirme	Sınırlı veya yok	Tam yönetim
Başarım	Genelde daha güçlü	Değişken
Destek	Profesyonel destek	Topluluk desteği
Güncelleme	Otomatik	Elle

BDM'lerin En Temel Görevi

BDM'lerin temelinde şaşırtıcı derecede basit ama güçlü bir kural yatar. Bu kural bir sonraki kelimeyi tahmin etmedir. Tüm o karmaşık konuşmalar, şiirler ve kod yazma işleri bu tek görev özelleştirilerek yapılır.

Bir sonraki kelimeyi tahmin etme

Örnek Cümle: "Köpek ___ peşinden koştu"

Model daha önce gördüğü benzer cümlelere dayanarak "*topun*" gibi bir kelimenin gelme ihtimalinin yüksek olduğunu tahmin eder.

Örnek seçenekler:

- *"topun" - %70 olasılık*
- *"kedinin" - %20 olasılık*
- *"arabanın" - %10 olasılık*

Olasılık dağılımı nasıl çalışır

Örnek Cümle: "Bugün hava çok ___"

Model her kelime için bir olasılık hesaplar:

- *"güzel" → 0.35*
- *"soğuk" → 0.25*
- *"sıcak" → 0.20*
- *"bulutlu" → 0.15*
- *"kötü" → 0.05*

Bu olasılıklardan birini seçerek cümleyi tamamlar.

Zincirleme tahmin

Uzun metinler nasıl oluşur?

Örnek Cümle: "Sabah erkenden ___"

- Model tahmin eder: "*kalktım*"
- Yeni durum: "Sabah erkenden *kalktım* ___"
- Model tahmin eder: "*ve*"
- "Sabah erkenden kalktım ve ___"
- Model tahmin eder: "*kahvaltı*"
- "Sabah erkenden kalktım ve kahvaltı ___"
- Model tahmin eder: "*yaptım*"

Bu şekilde kelime kelime devam ederek paragraflar, hatta sayfalar dolusu metin üretebilir.

Basit "Kelime Tahmini"nden Karmaşık Sorunları Çözmeye

Matematik Sorularını Çözme

Örnek: "5 + 3 = ?"

Model bu soruyu şöyle işler:

- "5 + 3 =" ifadesini görür
- Eğitim verisinde benzer kalıpları hatırlar
- "5 + 3 = 8" şeklinde tamamlar

Örnek: "Bir kutuda 12 elma var. 5 elma yedim. Kaç elma kaldı?"

Model'in tahmin zinciri:

- "12 elma var, 5 yedim, demek ki..."
- "12 - 5 ="
- "7"
- "Cevapta 7 elma kaldı"

Model aslında matematik yapmıyor...

Tercüme sorunu örneği

Örnek: "Translate to Turkish: The cat is sleeping"

Model'in tahmin süreci:

1. "Translate to Turkish: The cat is sleeping → Kedi"
2. "Kedi uyuyor"

Model, eğitim verisinde **milyonlarca** tercüme örneği görmüştür. Bundan dolayı eşleşmeleri başarılı şekilde yapabilmiştir

Soru-Cevap Sistemleri

Örnek: "Türkiye'nin başkenti neresidir?"

Model'in işleyişi:

- Soruyu tanır (soru kalıbı + "başkent" kelimesi)
- Eğitim verisinde benzer sorular ve cevapları vardır
- "Türkiye'nin başkenti Ankara'dır" şeklinde tamamlar

Bu başarının ana noktası ney?

- Kalıp tanıma: Model, dil kalıplarını tanır
  - "X'in başkenti" → şehir ismi gelir
  - "X + Y =" → sayı gelir
  - "Translate to X:" → çeviri gelir
- Bağlam anlama: Önceki kelimeler sonrakini belirler
  - "Matematik:" bağlamında sayılar ve işlemler
  - "Tarih:" bağlamında tarihler ve olaylar
- Kompozisyon yeteneği: Öğrendiği parçaları birleştirir
  - Gramer kuralları + kelime bilgisi = yeni cümleler
  - Problem kalıpları + sayılar = matematik çözümleri

Ön-Eğitim (Pre-training) ve İnce Ayar (Fine-tuning) Kavramlarına Genel Bakış

Ön-eğitim (pre-training)

Bir BDM'nin "beyninin oluşturulduğu" en önemli aşamadır. Bu süreçte model, devasa miktardaki metni işleyerek dil ve dünya bilgisini parametrelerine sıkıştırır.

Modelin herhangi bir özel göreve odaklanmadan, sadece "bir sonraki kelimeyi tahmin et" göreviyle eğitilmesidir.

Bu süreçte

- İnternetin büyük bölümü (Wikipedia, kitaplar, makaleler, kod depoları)
- Milyarlarca hatta trilyonlarca kelime
- Aylar süren eğitim
- Binlerce GPU

kullanımı söz konusudur

Neden "sıkıştırma" diyoruz?

Bakıldığında:

- Girdi: 10 TB'lık metin verisi (milyarlarca web sayfası)
- Çıktı: 70 GB'lık model dosyası
- Model, 140 kat sıkıştırma yaparak tüm bu bilgiyi parametrelerine kodluyor. Tıpkı bir ZIP dosyası gibi, ama kayıplı sıkıştırma - her ayrıntıyı değil, bilginin özütünü saklıyor.

Ön-Eğitim Süreci

1. Veri Toplama ve Temizleme
   - İnternet → Filtreler → Temiz metin
2. Tokenizasyon
   - "Merhaba dünya" → [Mer, haba, dün, ya] →[312321, 5321, 5, 9]
3. Eğitim

İnce Ayar (Fine-Tuning)

Ön-eğitimli bir modeli özel bir göreve uyarlamak için yapılan ikinci eğitim aşamasıdır. Bu süreçte model, daha az ama yüksek kaliteli veriyle belirli bir işi yapabilir hale gelir.

• Başlangıç: Ön-eğitimli model (genel bilgi sahibi)

- Hedef: Özel görev uzmanı (asistan, kod yazıcı, çevirmen)
- Yöntem: Az miktarda güzel veriyle ek eğitim

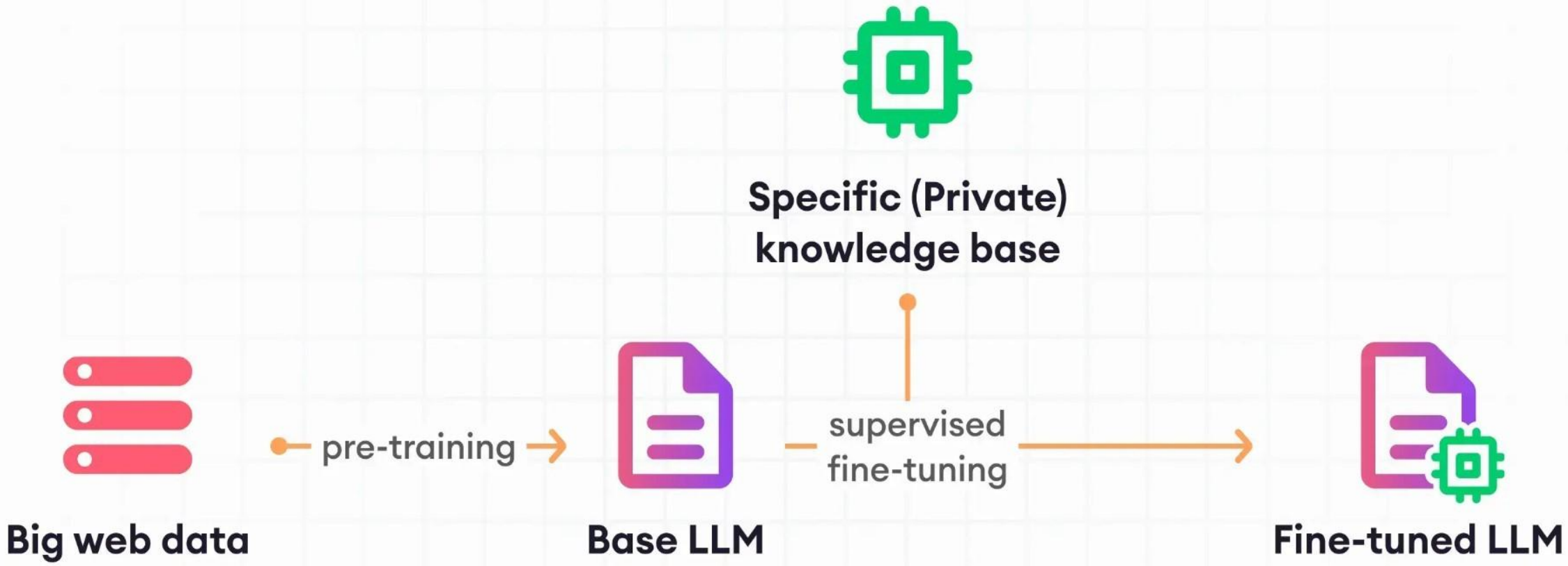
İnce Ayar Türleri

- Supervised Fine-Tuning (SFT)
  - Örnek veri:

    {İnsan: "Python'da liste nasıl oluşturulur?"
    Asistan: "Python'da liste oluşturmak için köşeli parantez kullanabilirsiniz:
    my_list = [1, 2, 3, 4, 5]"}
- Instruction Tuning Model
  - Model talimatları takip etmeyi öğrenir.
  - {Talimat: "Bu metni özetle"} gibi
- RLHF (Reinforcement Learning from Human Feedback)
  - İnsan değerlendiriciler cevapları puanlar
  - Model, yüksek puan alan davranışları öğrenir

İnce Ayar - Ön Eğitim Karşılaştırması

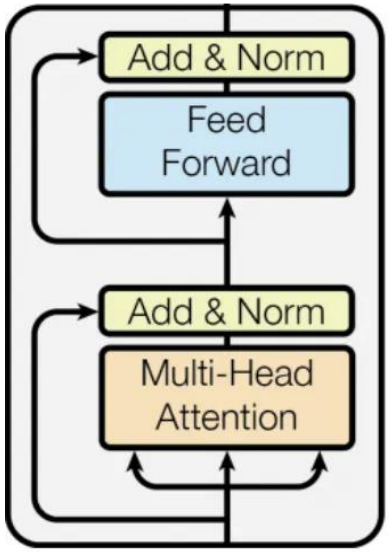
Özellik	Ön-Eğitim	İnce Ayar
Veri miktarı	Trilyonlarca token	Binlerce-milyonlarca token
Veri kalitesi	Değişken (internet)	Çok yüksek (özenle seçilmiş)
Süre	Aylar	Saatler-günler
Maliyet	Milyonlarca dolar	Yüzlerce dolar
Amaç	Genel dil anlayışı	Özel görev başarımı



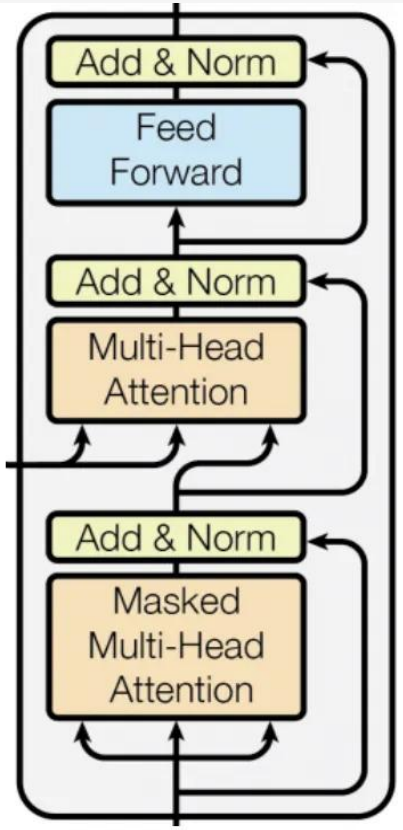
Kaynak: https://dzone.com/articles/llm-fine-tuning-strategies-for-domain-specific-app

Transformer Mimarisine Kuşbakışı

Transformer mimarisi, 2017'de Google tarafından tanıtılan ve çağdaş BDM'lerin temelini oluşturan devrim niteliğindeki bir yapıdır Bu mimari, üç farklı kullanılabilir: Encoder, Decoder ve Encoder-Decoder.



Encoder layer



Decoder layer

Kaynak: https://cerebralvalley.ai/blog/how-do-transformers-work-really-5WyLp7VulK1vuh9u7YE21f

Encoder Yapısı

Giriş metnini alıp anlamlı temsillere dönüştüren bileşendir.

Temel görevleri:

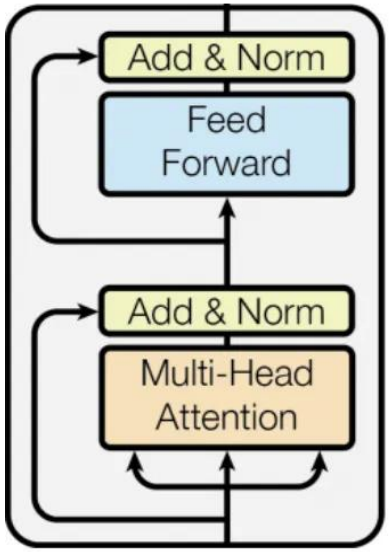
- Metni "anlama" ve kodlama
- Her kelimenin bağlamsal temsilini oluşturma
- Giriş dizisini zengin özellik vektörlerine dönüştürme

Örnek Modeller:

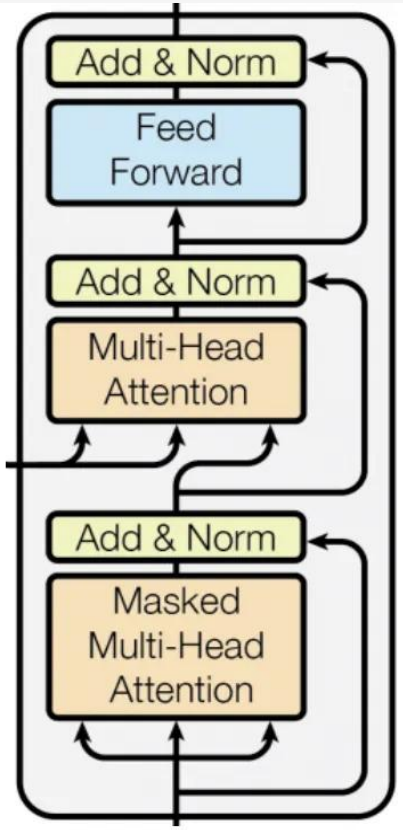
- BERT (Bidirectional Encoder Representations from Transformers)
- RoBERTa
- ELECTRA

Bu mimariye sahip modeller bilhassa anlama görevlerinde başarılıdır:

- Metin sınıflandırma
- Duygu analizi
- İsimlendirilmiş varlık tanıma (NER)



Encoder layer



Decoder layer

Kaynak: https://cerebralvalley.ai/blog/how-do-transformers-work-really-5WyLp7VulK1vuh9u7YE21f

Decoder Yapısı

Metin üretmeye odaklanan yapıdır. Önceki kelimelere bakarak bir sonraki kelimeyi tahmin eder.

Temel görevleri:

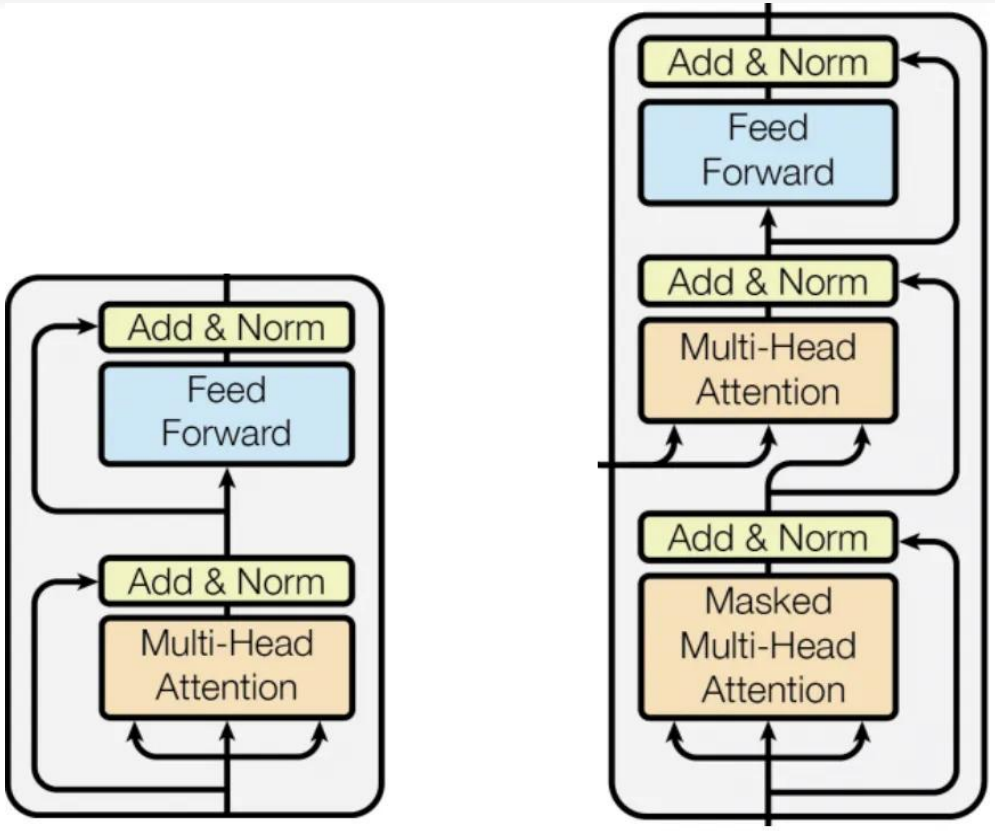
- Soldan sağa metin üretimi
- Otoregresif tahmin (her kelime öncekine bağlı)
- Benzersiz içerik oluşturma

Örnek Modeller:

- GPT serisi (GPT-2, GPT-3, GPT-4)
- Claude
- LLaMA

Bu mimariye sahip modeller bilhassa anlama görevlerinde başarılıdır:

- Metin tamamlama
- Hikaye yazma
- Kod üretimi
- Sohbet sistemleri



Encoder layer

Decoder layer

Kaynak: https://cerebralvalley.ai/blog/how-do-transformers-work-really-5WyLp7VulK1vuh9u7YE21f

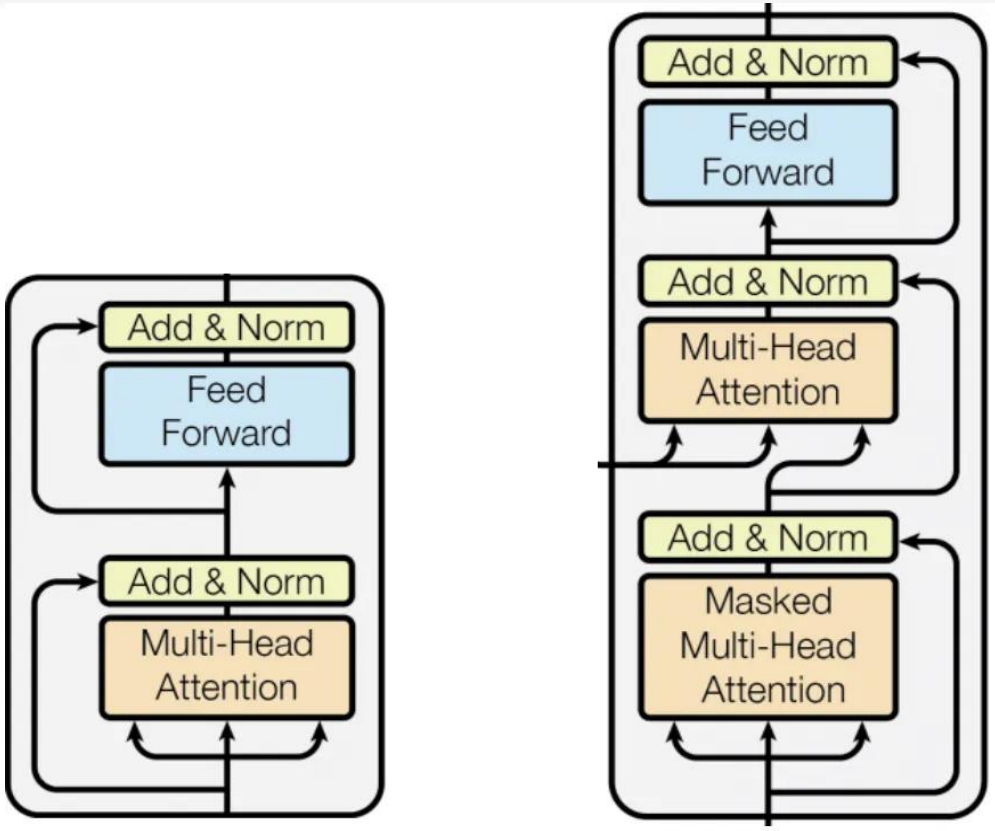
Encoder-Decoder Yapısı

Her iki bileşeni içeren karışık yapıdır:
Enkoder kısmı: Girişi anlar ve kodlar
Dekoder kısmı: Enkoder'ın çıktısını kullanarak yeni metin üretir
Örnek Modeller:

- T5
- BART
- mT5

Bu mimariye sahip modeller bilhassa anlama görevlerinde başarılıdır:

- Makine tercümesi
- Metin özetleme
- Soru cevaplama
- Metin düzeltme



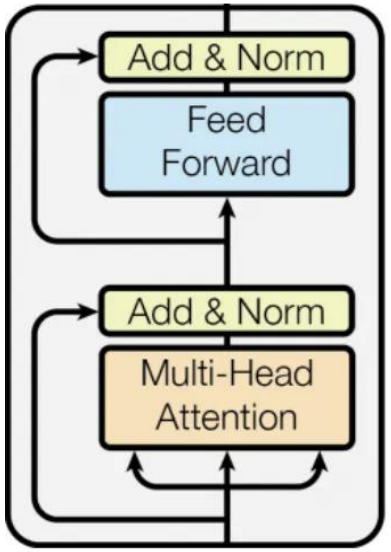
Encoder layer Decoder layer

Kaynak: https://cerebralvalley.ai/blog/how-do-transformers-work-really-5WyLp7VulK1vuh9u7YE21f

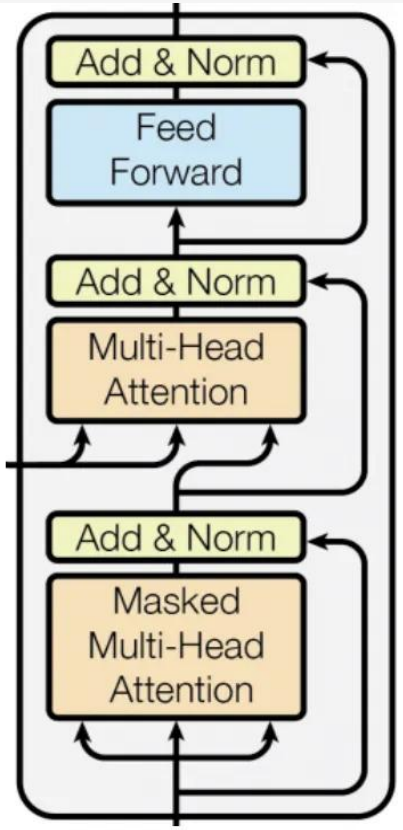
Encoder-Decoder

Encoder = Dinleyici/Okuyucu
Decoder = Yazar/Konuşmacı
Encoder-Decoder = Tercüman
Son yıllarda yalnızca **decoder** mimarisine sahip modeller (GPT gibi) popülerlik kazandı çünkü:

- Tek bir yapıyla hem anlama hem üretim
- Daha basit eğitim süreci
- Ölçeklendirmesi daha kolay
- Genel amaçlı kullanıma uygun



Encoder layer



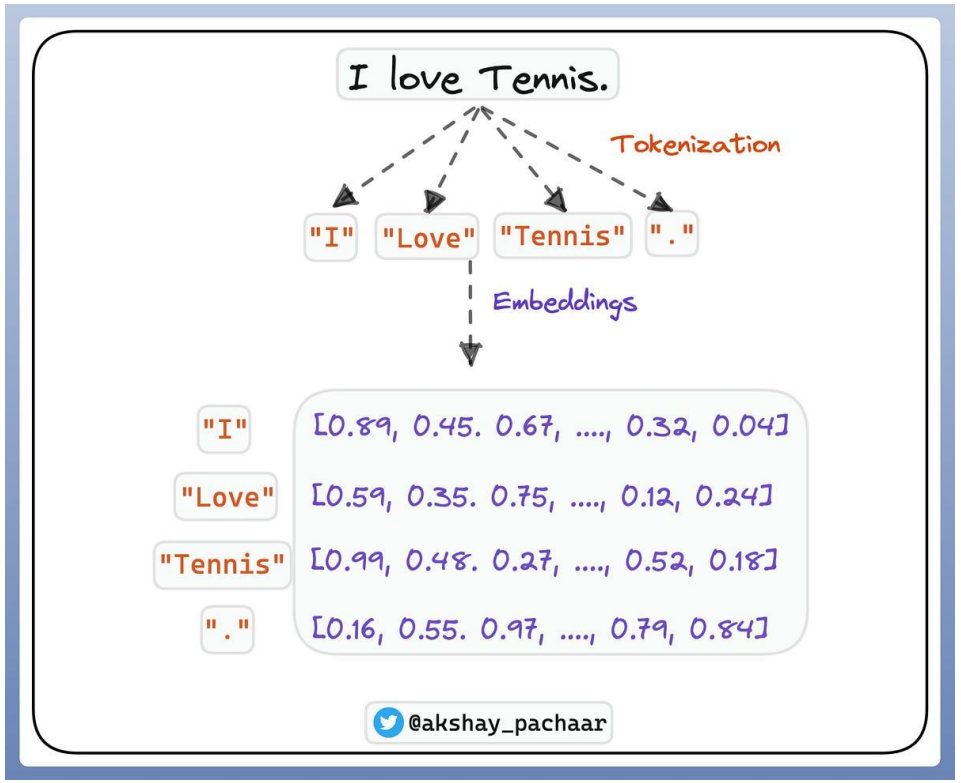
Decoder layer

Kaynak: https://cerebralvalley.ai/blog/how-do-transformers-work-really-5WyLp7VulK1vuh9u7YE21f

Dilin Yapıtaşları: Tokenization Süreci

Tokenizer Nedir ve Neden Gereklidir?

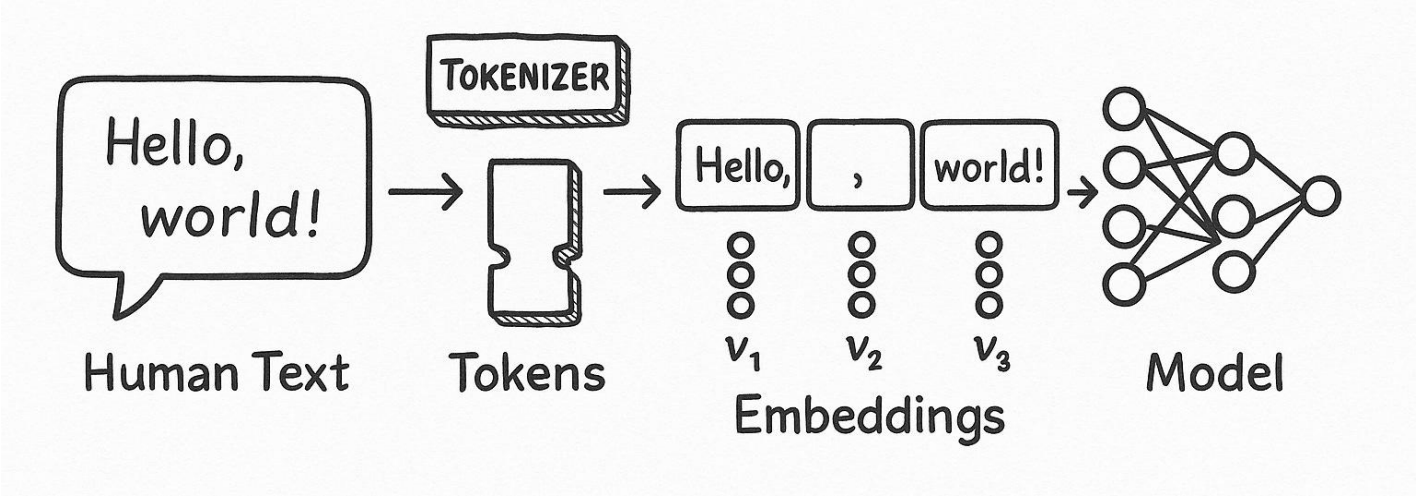
Tokenizer, metin ile matematik arasındaki köprüdür. LLM'lerin en temel gerçeği, bilgisayarlar ve sinir ağlarının metinlerle değil sadece sayılarla çalıştığıdır. İşte tokenizer'ın görevi, bu dönüşümü sağlamaktır.



Modeller Neden Sayılara İhtiyaç Duyar?

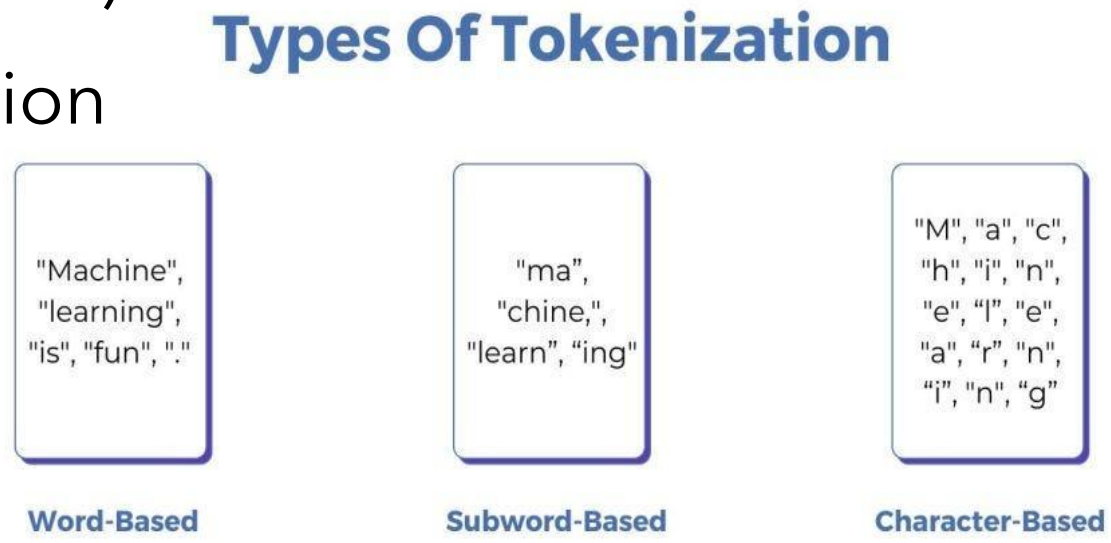
Sinir ağları sadece matematiksel işlemler yapabilir:

- Matris çarpımları
- Vektör işlemleri
- Gradyan hesaplamaları
- Aktivasyon fonksiyonları



Tokenization Algoritmaları

- Kelime Tabanlı (Word-based) Tokenizer'lar
- Karakter Tabanlı (Character-based) Tokenizer'lar
- Alt Kelime (Subword) Tokenization



Kelime Tabanlı (Word-based) Tokenizer'lar

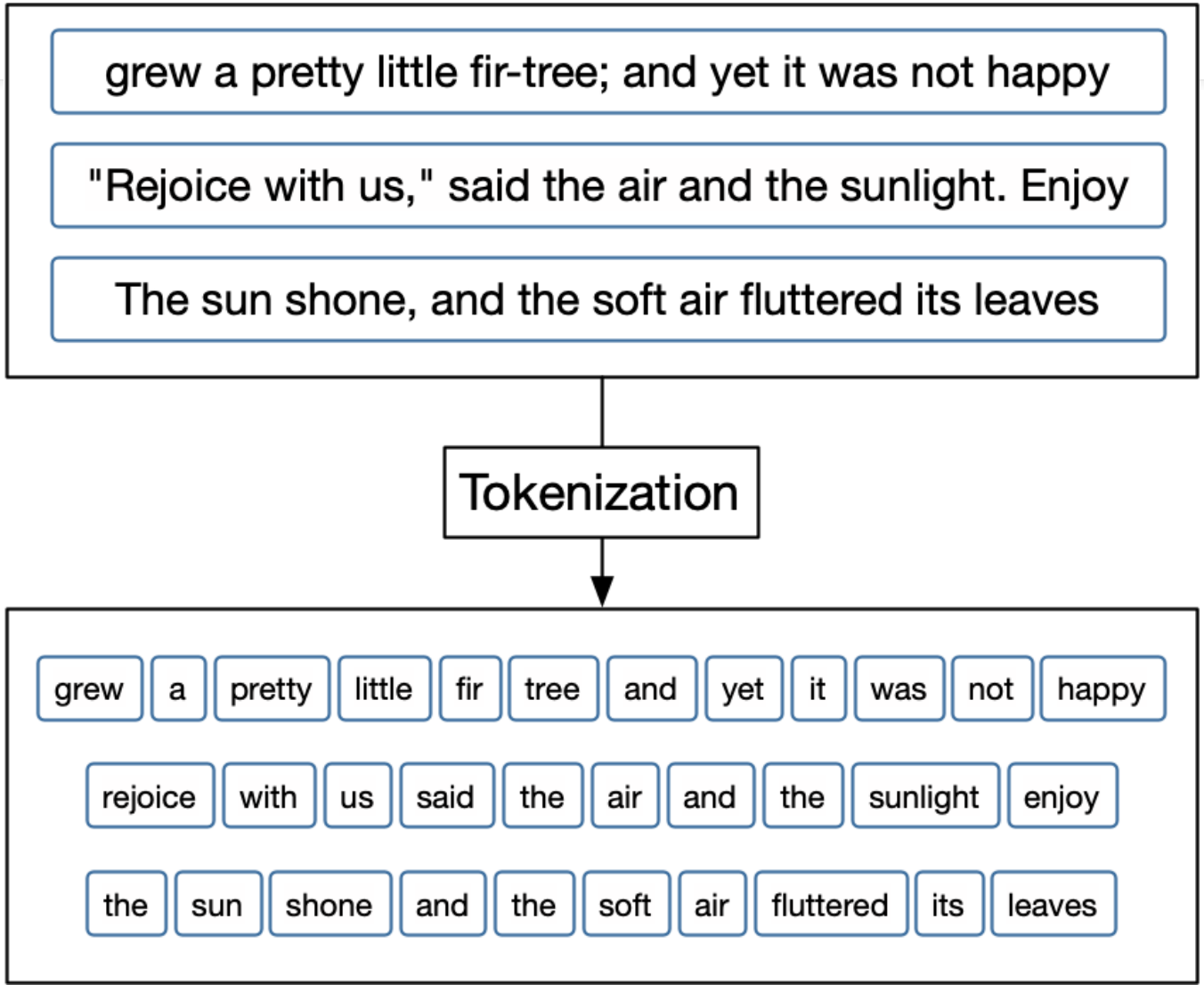
Artıları:

- Anlaşılması kolay
- Anlamlı birimler (tam kelimeler)
- Hızlı işleme

Eksileri:

- Devasa kelime haznesi (100,000+ kelime)
- OOV (Out-of-Vocabulary) sorunu: Eğitimde görülmeyen kelimeler işlenemez
- Farklı kelime formları ayrı token'lar ("yap", "yaptı", "yapacak")
- Yazım hataları ve yeni kelimelerle başa çıkamaz

Kelime Tabanlı (Word-based) Tokenizasyon



Karakter Tabanlı (Character-based) Tokenizer'lar

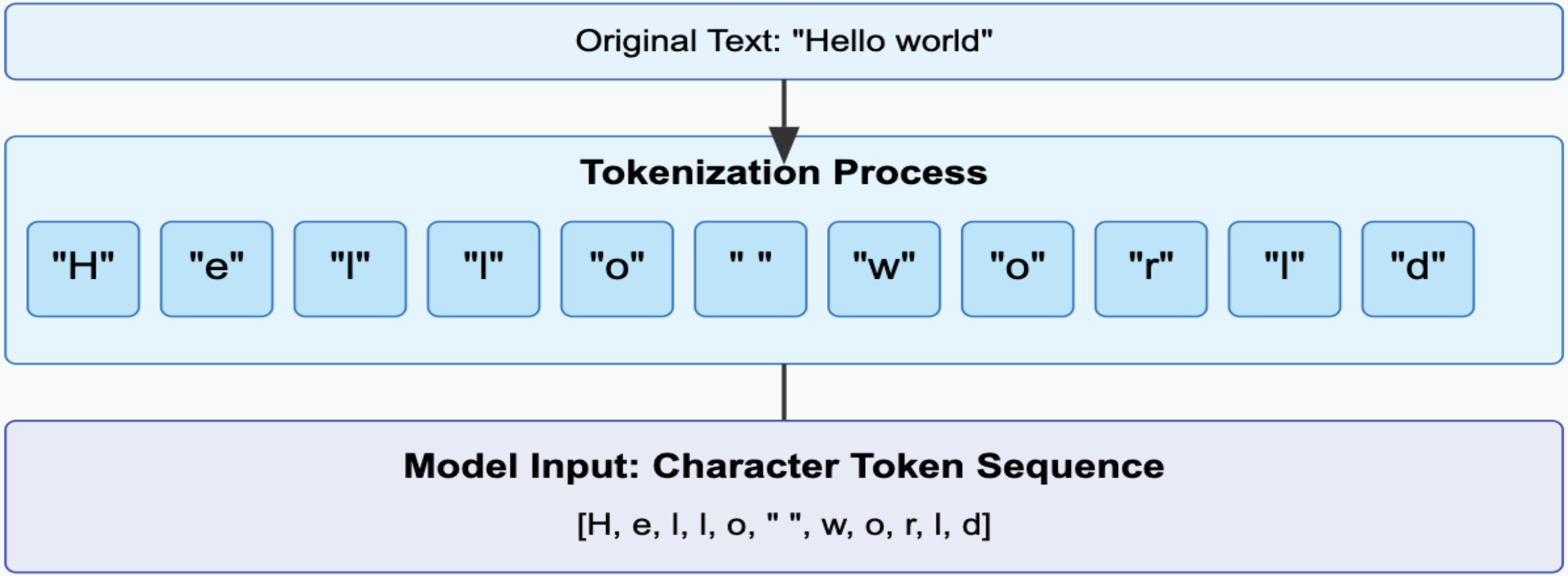
Artıları:

- Çok küçük kelime haznesi (genelde <100 karakter)
- OOV sorunu yok (her kelime karakterlerden oluşur)
- Yazım hatalarına dayanıklı
- Tüm dillerde çalışır

Eksileri:

- Çok uzun diziler (her kelime onlarca token)
- Anlam kaybı (karakterler tek başına anlamsız)
- Yavaş işleme (çok fazla token)
- Model için öğrenmesi zor

Character-Based Tokenization



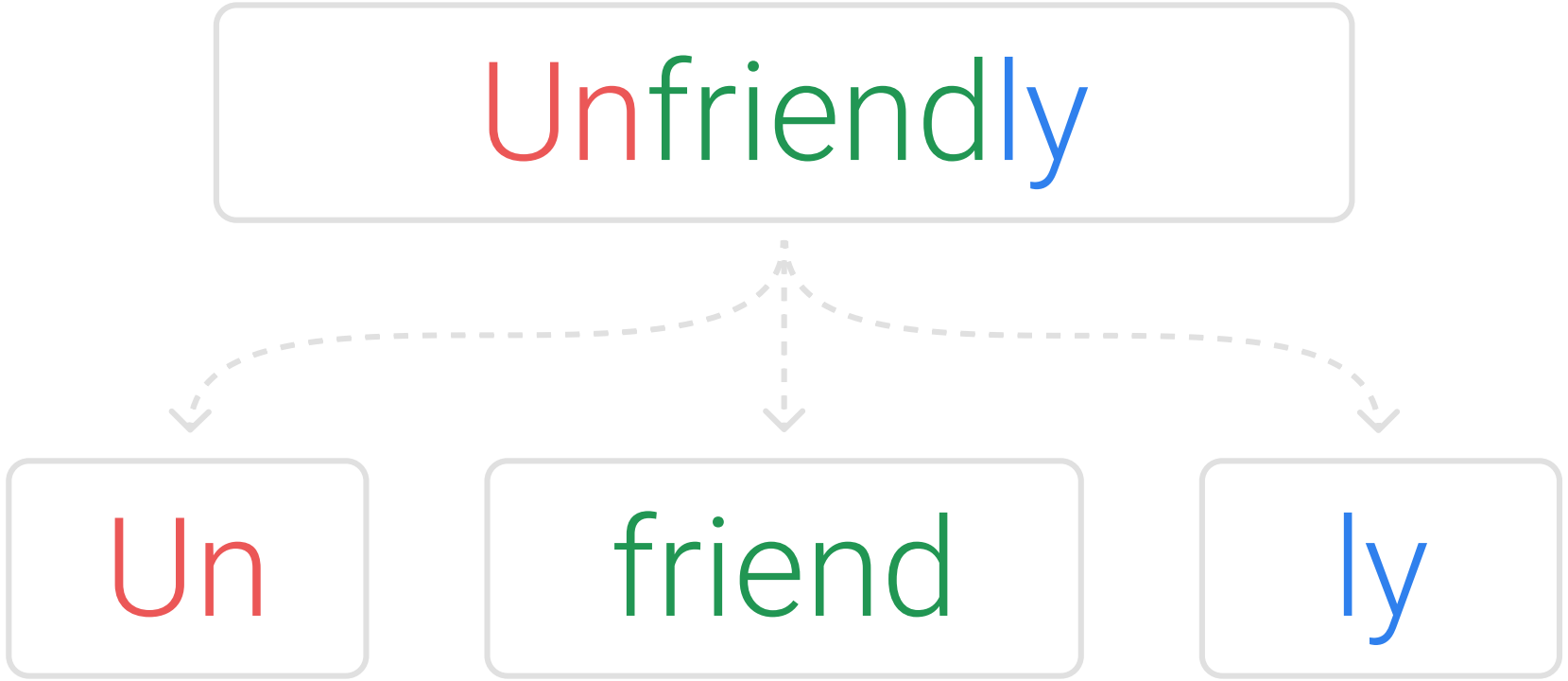
Kaynak: https://www.linkedin.com/pulse/llm-tokenizers-hidden-engine-behind-ai-language-models-natarajan-rmbsc/

Alt Kelime (Subword) Tokenization

Artıları:

- Optimal kelime haznesi (genelde 30,000-50,000)
- OOV sorunu minimal
- Verimli temsil
- Morfolojik yapıyı korur
- Farklı dillerde etkili

Alt Kelime (Subword) Tokenization



Tokenizasyon örneği

Tokens	Characters
94	382

We're no strangers to love
You know the rules and so do I (do I)
A full commitment's what I'm thinking of
You wouldn't get this from any other guy
I just wanna tell you how I'm feeling
Gotta make you understand
Never gonna give you up
Never gonna let you down
Never gonna run around and desert you
Never gonna make you cry
Never gonna say goodbye
Never gonna tell a lie and hurt you

Text Token IDs

Neden Doğrudan Harfler veya Kelimeler Kullanmıyoruz?

Harf seviyesi sıkıntıları

- Çok uzun diziler (her kelime onlarca harf)
- Anlam kaybı
- Verimsiz işleme

Kelime seviyesi sıkıntıları

- Devasa sözlük boyutu (milyonlarca farklı kelime)
- Bilinmeyen kelime sorunu
- Farklı dillerde uyumsuzluk

Alt kelime seviyesi (ideal)

- Dengeli sözlük boyutu
- Hem yaygın kelimeleri hem alt-parçaları içerir
- Bilinmeyen kelimelerle başa çıkabilir

Yaygın Alt Kelime (Subword) Algoritmaları

- Byte-Pair Encoding (BPE)
- WordPiece

Byte-Pair Encoding (BPE)

1994'te Philip Gage tarafından metin sıkıştırma için geliştirilmiş, sonradan NLP'de popüler olmuş bir algoritmadır. **Temel mantığı**, en sık tekrar eden karakter çiftlerini birleştirmektir.

Örnek: "hugging" kelimesini tokenize edelim.

- Başlangıç: h u g g i n g
- Adım 1: En sık çift "gg" → h u gg i n g
- Adım 2: En sık çift "in" → h u gg in g
- Adım 3: En sık çift "hugg" → hugg in g
- Adım 4: En sık çift "ing" → hugg ing
- Sonuç: ["hugg", "ing"]

WordPiece

Google tarafından BERT için geliştirilmiş, BPE'nin bir türüdür. BPE'den temel farkı, birleştirme kriterinde olasılık tabanlı yaklaşım kullanmasıdır.

BPE sadece frekansa bakarken, WordPiece likelihood (olabilirlik) hesaplar. Yani:

$$Skor = freq(AB) / (freq(A) \times freq(B))$$

En yüksek skoru alan çift birleştirilir.

Hugging Face Fast Tokenizer

Hugging Face'in Fast Tokenizer'ları, geleneksel Python tokenizer'lardan 100 kata kadar daha hızlı çalışır. Bu inanılmaz hız artışının sebebi kodunun Rust programlama diliyle paralelleştirme yapılarak yazılmış olmasıdır.

	Fast tokenizer	Slow tokenizer
`batched=True`	10.8s	4min41s
`batched=False`	59.2s	5min3s

Kaynak: https://www.digitalocean.com/community/tutorials/run-tokenizer-on-gpu-for-faster-nlp

LLM'lerin Beyni: Transformer Mimarisi

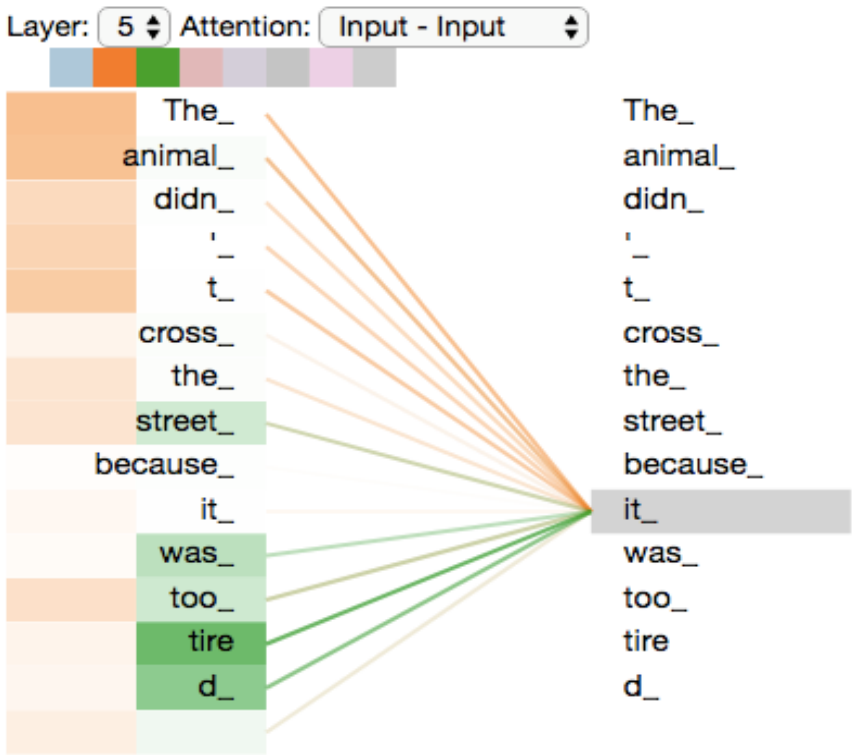
Öz-Dikkat (Self-Attention) İşleyişine Sezgisel Bir Bakış

Öz-Dikkat (Self-attention), Transformer mimarisinin kalbi olan ve BDM'lerin bu kadar başarılı olmasını sağlayan devrim niteliğindeki işleyiştir. Temel olarak, bir kelimenin anlamının cümledeki diğer kelimelerle olan alakasına göre şekillenmesine vesile olur.

Öz-Dikkat (Self-Attention) Nedir?

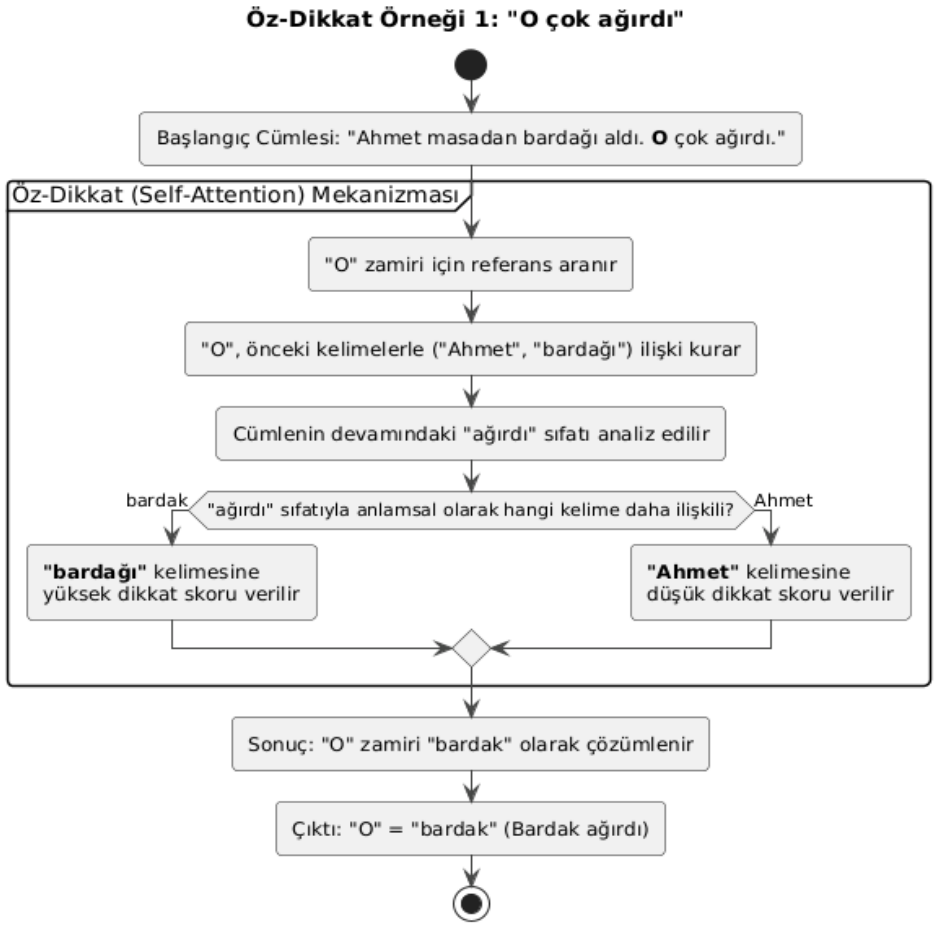
Öz-Dikkat (Self-attention) mekanizması, metin içindeki her bir kelimenin diğer tüm kelimelerle nasıl alakada olduğunu hesaplar. Bu vesileyle model, kelimelerin bağlamsal anlamlarını yakalayabilir.

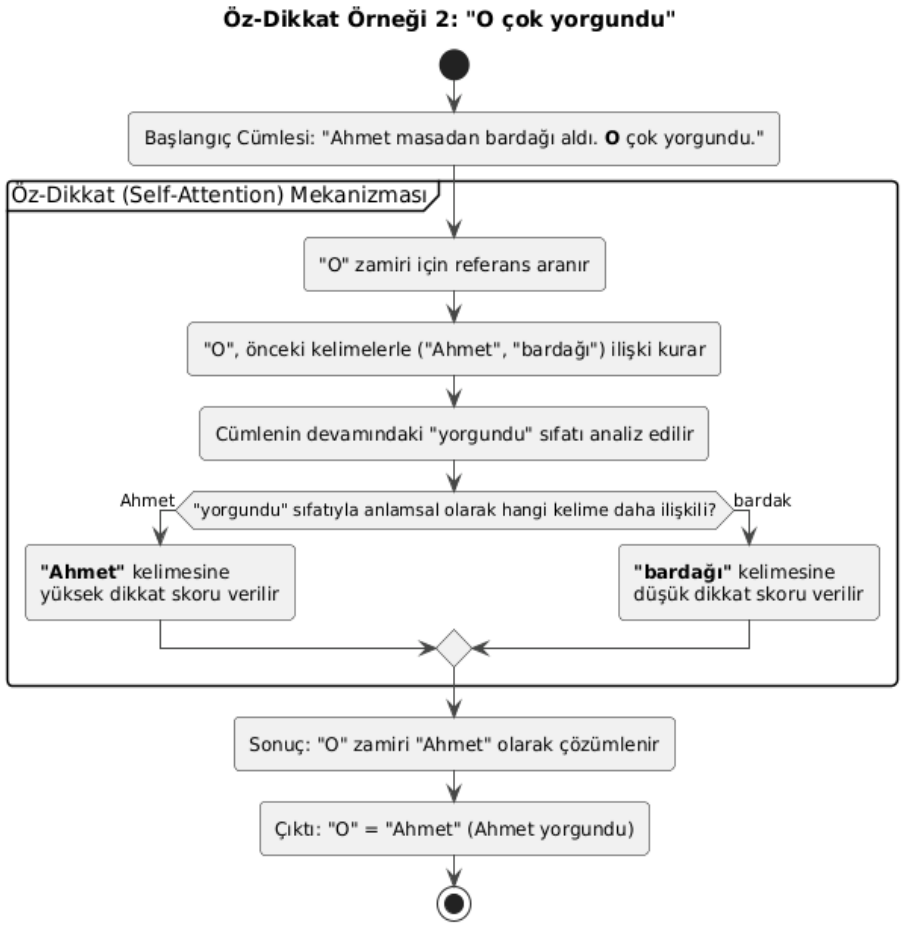
Basit Tanım: Her kelime, cümledeki diğer tüm kelimelere "bakar" ve hangilerinin kendisi için önemli olduğuna karar verir.



kaynak: https://jalammar.github.io/illustrated-transformer/

Öz-Dikkat (Self-Attention) Örnek





Öz-Dikkat'in (Self-Attention) Faydaları

1. **Paralel İşleme:** RNN'lerin aksine, tüm kelimeler aynı anda işlenebilir.
2. **Uzun Mesafe İlişkileri:** Cümlenin başındaki kelime ile sonundaki kelime arasındaki bağlantıyı kolayca kurabilir.
3. **Bağlamsal Anlam:** Aynı kelime, farklı bağlamlarda farklı temsiller alır.

Encoder Mimarisi BERT ve Türevi Modellerin Gücü

Encoder mimarisi, metni derinlemesine "anlamak" için tasarlanmış transformer yapısıdır. BERT (Bidirectional Encoder Representations from Transformers), bu mimarinin en ünlü temsilcisi olarak 2018'de Google tarafından tanıtılmış ve DDİ dünyasını sarsmıştır.

Encoder'ın Temel Görevi

Bu görev iki kelimeyle "Metni Anlamak" olarak ifde edilebilir.

Encoder modelleri, metni alıp zengin, bağlamsal temsiller (embeddings) oluşturur. Decoder'lardan farklı olarak yeni metin üretmezler, bunun yerine mevcut metni derinlemesine incelemeye çalışır.

BERT ise, çok katmanlı bidirectional (çift yönlü) transformer encoder kullanarak giriş metnini yüksek boyutlu bir uzayda temsil eder. Bu vesileyle cümledeki her kelimenin tüm bağlamını hesaba katabilir.

BERT Tahmin Örneği

- **Örnek:** "Banka hesabımı kapattım"
- BERT Encoder İşlemi:
- - "Banka" kelimesi hem önceki hem sonraki kelimelere bakar
- - Finans kurumu olduğunu anlar (park bankası değil)
- - Her kelime için 768 boyutlu vektör üretir
- **Çıktı:** Zengin bağlamsal temsiller

Örnek: "Kedim süt [MASK] çok severdi" → Hem önceki hem sonraki kelimelere bakar

Sonuç: "içmeyi" kelimesini daha doğru tahmin eder

Zengin Temsiller Nasıl Oluşur?

BERT, her kelime için bağlama duyarlı vektör temsilleri oluşturur.

1. **Token Embedding:** Kelimenin temel anlamı.
2. **Position Embedding:** Kelimenin cümledeki konumu.
3. **Segment Embedding:** Hangi cümleye ait olduğu.
4. **Self-Attention Katmanları:** 12 veya 24 katmanda bağlamsal zenginleştirme.

Kullanım Alanı 1: Metin Sınıflandırma

Duygu Analizi Örneği:

Giriş: "Bu film harikaydı, kesinlikle tavsiye ederim!"

BERT İşlemi:

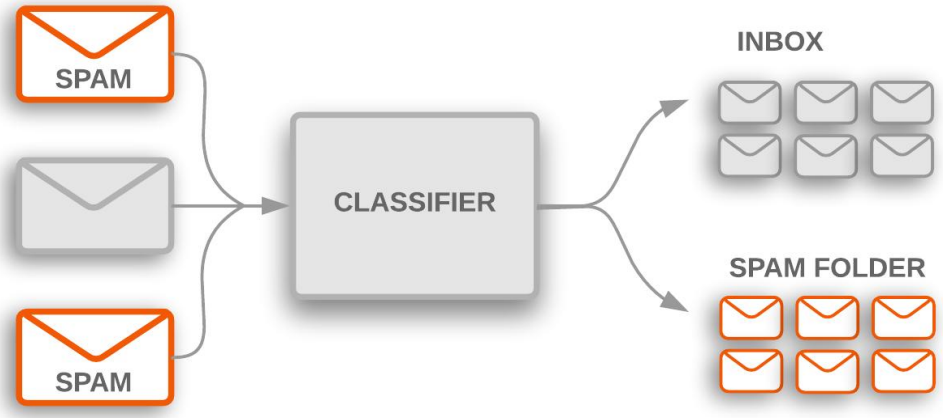
1. *Her kelime için bağlamsal vektör oluştur*
2. *[CLS] token'ının temsilini al (cümle özeti)*
3. *Sınıflandırma katmanına gönder*

Çıktı: Olumlu (0.95 güven skoru)

Spam Tespiti Örneği:

"Tebrikler! 1 milyon TL kazandınız, hemen tıklayın!"

BERT → Spam (0.98 güven)



Kullanım Alanı 2: Varlık İsmi Tanıma (NER)

NER Örneği:

Giriş: "Ahmet İstanbul'da Google'da çalışıyor"

BERT İşlemi:

"Ahmet" → [PER] (Kişi)

"İstanbul" → [LOC] (Yer)

"Google" → [ORG] (Organizasyon)

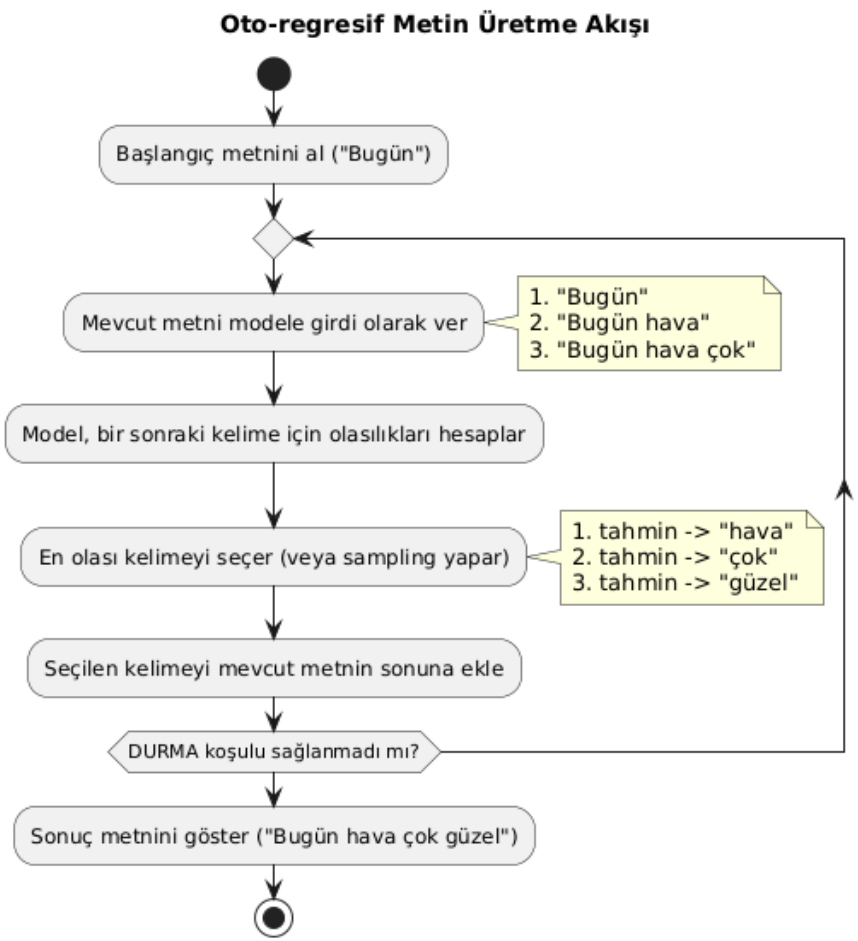
"çalışıyor" → [O] (Varlık değil)

Decoder Mimarisi: GPT ve Türevi Modellerle Metin Üretimi

Decoder mimarisi, çağdaş BDM'lerin metin üretme yeteneğinin arkasında yatan mimaridir. GPT (Generative Pre-trained Transformer) serisi, bu mimarinin en başarılı örnekleridir ve **oto-regresif** yaklaşımla metin üretirler.

Oto-regresif (Auto-regressive) Metin Üretme Mantığı

Modelin her yeni kelimeyi **önceki tüm kelimelere** bağlı olarak üretmesi anlamına gelir. Yani her adımda, o ana kadar üretilen metni girdi olarak alıp bir sonraki kelimeyi tahmin eder.



Maskeli Öz-Dikkat (Masked Self-Attention)

Decoder'ın en mühim özelliği maskeli öz-dikkat işleyişidir. Bu işleyiş, modelin "geleceği görmesini" ve kopya çekmesini engeller.

Eğitim sırasında model tüm cümleyi görür:

"Kedi süt içti ve uyudu"

"içti" kelimesini tahmin ederken, model "uyudu" kelimesini görmemeli. Çünkü gerçek kullanımda henüz o kelime üretilmemiş olacak!

Maskeleme Nasıl Çalışır?

Cümle: "Kedi süt içti"

Position 1 (Kedi): [Kedi, -, -] → Sadece kendini görür

Position 2 (süt): [Kedi, süt, -] → İlk iki kelimeyi görür

Position 3 (içti): [Kedi, süt, içti] → Tüm önceki kelimeleri görür

	Kedi	süt	içti
Kedi	✓	X	X
süt	✓	✓	X
içti	✓	✓	✓

✓ = Görebilir, X = Maskelenmiş

Decoder'ın Güçlü Yanları

1. Benzersiz *Üretim*
    a. *Hiç görmediği cümleleri üretebilir*
    b. *Farklı tarzlar ve sertliklerde yazabilir*
2. Bağlamsal Tutarlılık
    a. *Uzun metinlerde bile tutarlılığı korur*
    b. *Önceki* bağlamı unutmaz
3. Çok Amaçlı Kullanım
    a. Kod *üretimi*
    b. *Hikaye yazma*
    c. *Diyalog sistemleri*
    d. *Tercüme*

Temperature

Decoder'lar farklı özgünlük seviyeleri ile metin üretebilir.

Temperature = 0.1 (Daha Belirli)

"Güneş doğudan doğar" → Her zaman aynı devam

Temperature = 1.0 (Dengeli)

"Güneş doğudan" → "doğar" / "yükselir" / "görünür"

Temperature = 2.0 (Özgün/Tehlikeli):

"Güneş doğudan" → "dans eder" / "şarkı söyler"

Encoder-Decoder (Sequence-to-Sequence) Mimarisi

İki güçlü transformer yapısını birleştirerek sequence-to-sequence (seq2seq) görevlerde yüksek başarım sağlar.

Bu mimari, bir diziyi başka bir diziye dönüştürmek için tasarlanmıştır ve encoder kaynak metni anlarken, decoder hedef metni üretir.

İki Yapının Mükemmel Birleşimi

Encoder-Decoder mimarisi, iki aşamalı bir süreç izler:

1. **Enkoder Aşaması - Anlama**
   - Kaynak metin → Encoder → Bağlamsal temsil (context vector)
   - "Hello world" → [Encoding işlemi] → Zengin anlam vektörü
2. **Dekoder Aşaması - Üretim**
   - Bağlamsal temsil → Decoder → Hedef metin
   - Anlam vektörü → [Decoding işlemi] → "Merhaba dünya"

Adım Adım İşleyiş:

1. Kaynak metni tokenize et. "Türkiye büyük bir devlet." → [101, 2023, 3456 ,234, 7543]
2. Encoder işlemi: Her token için bağlamsal temsiller oluştur.
3. Context vector oluştur: Tüm encoder çıktılarını özetle.
4. Decoder'a aktar: Context vector decoder'ın başlangıç durumu olur.
5. Oto-regresif üretim: Decoder hedef metni kelime kelime üretir.

Kullanım Alanı 1: Makine Tercümesi

Mimari, makine tercümesi için ideal bir yapıdır. Kaynak dili anlayıp hedef dilde yeniden üretme mantığı, tercüme görevi için biçilmiş kaftandır.

Örnek:

1. İngilizce (Kaynak): "The weather is beautiful today"
2. Encoder: Cümlenin anlamını yakalar
3. Context Vector: Dil-bağımsız anlam temsili
4. Decoder: Türkçe dilbilgisi kurallarıyla yeniden üretir
5. Türkçe (Hedef): "Bugün hava çok güzel"

Kullanım Alanı 2: Metin Özetleme

Örnek:

1. Uzun Metin (500 kelime):
   - "Yapay zeka, son yıllarda hızla gelişen bir teknoloji alanıdır. Machine learning ve deep learning teknikleri sayesinde..."
2. Encoder: Metnin ana fikirlerini çıkarır
3. Decoder: Önemli noktaları içeren özet üretir
4. Özet (50 kelime):
   - "Yapay zeka, ML ve DL teknikleriyle hızla gelişmektedir..."

Encoder, Decoder, Encoder-Decoder Kıyaslama

Özellik	Encoder (BERT)	Decoder (GPT)	Encoder-Decoder (T5/BART)
Temel Amaç	Metni anlamak ve temsil etmek	Metin üretmek	Metni dönüştürmek
Attention Türü	Bidirectional (çift yönlü)	Causal/Masked (tek yönlü)	Encoder: Bidirectional Decoder: Causal + Cross-attention
Girdi-Çıktı	Sabit boyutlu temsil	Değişken uzunlukta metin	Değişken girdi → Değişken çıktı
Eğitim Görevi	Masked Language Modeling (MLM)	Next Token Prediction	Denoising + Seq2Seq
Parametre Sayısı	Orta (110M-340M)	Büyük (175B'ye kadar)	En büyük (2x model)
Hız	Hızlı (paralel işleme)	Yavaş (oto-regresif)	En yavaş (iki aşamalı)
Güçlü Olduğu Görevler	• Metin sınıflandırma • NER • Duygu analizi • Soru-cevap	• Metin üretimi • Kod yazma • Hikaye anlatımı • Sohbet	• Tercüme • Özetleme • Metin düzeltme • Paraphrase
Zayıf Olduğu Görevler	Metin üretimi	Hassas anlama görevleri	Serbest üretim
Popüler Modeller	BERT, RoBERTa, ELECTRA	GPT-3/4, Claude, LLaMA	T5, BART, mT5
Bağlam Penceresi	512-1024 token	2K-128K token	Girdi+Çıktı toplamı sınırlı
Fine-tuning Kolaylığı	Çok kolay	Orta	Zor (iki model)
Bellek Kullanımı	Düşük	Orta	Yüksek
Inference Maliyeti	Düşük	Yüksek	En yüksek
Zero-shot Yeteneği	Zayıf	Çok güçlü	Orta
Çok Dilli Destek	İyi (mBERT)	Çok iyi	Mükemmel (mT5)

TEŞEKKÜRLER